

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

Last Week's Lecture

Backend Integration

Support dynamic, multi-user, persistent apps

- Options:
- **REST APIs**: Public, Mock, Custom
- **BaaS**: Ready-to-use infrastructure (Firebase, Supabase)
- **Supabase** Setup: Use [@supabase/supabase-js](#)
- Store public API key as environment variable
- Type-safe queries with generated TypeScript types

Last Week's Lecture

User Authentication

- **Flow:** Sign Up → Sign In → Authenticated Requests → Logout
- **Token:** JWT for stateless authentication
- **Options:** Firebase Auth, Supabase Auth, Expo LocalAuthentication
- Combine Auth with CRUD: Supabase handles auth + database in one place
 - Example: Attach `user.id` to user-specific data
 - Queries filtered by current user

Today's Lecture

Deploying Mobile Apps with Expo

Why Deployment Matters

- Turns your project from **code on your laptop** → **installable app**
- Allows others (TAs, classmates, testers) to experience your work
- Course Project Requirement
 - Build and deploy the app using **Expo EAS Build**
 - Provide a **testable build** (APK, IPA, or Expo Go link)

Expo EAS: Overview

Expo Application Services (EAS): Expo's cloud service for building, submitting, and updating Expo and React Native apps

- **EAS Build**: Build **native binaries** ([.ipa](#) for iOS, [.apk](#) / [.aab](#) for Android) in Expo's cloud servers
 - No need for local Xcode or Android Studio
 - After the build completes, you can download the **standalone native app binary**, install it on a device, or use it for store submission

EAS: Three Services

EAS Build: Build **native binaries**

EAS Submit: Automate app store submissions

- Take your built binary (from EAS Build) and upload it automatically to App Store or Google Play
- You provide store credentials or API keys once
- EAS handles the submission pipeline

EAS Build → EAS Submit → App Store / Google Play

EAS: Three Services

EAS Build: Build **native binaries**

EAS Submit: Automate app store submissions

EAS Update: Push updates directly to users

- Work after your app is installed (App Store/Google Play)
- Let you push JavaScript/asset updates (not native code)
- Users automatically get the latest JS bundle when they reopen app

Pipeline

•

Expo EAS Build

Expo Application Services Build

- Cloud service that compiles your Expo/React Native app
- Produces installable native binaries
- iOS: `.ipa`
- Android: `.apk` or `.aab`
- No need for local Xcode or Android Studio
- **Free Tier**: 15 Android and 15 iOS builds/month

EAS: Setup

Official Doc: **Create Your First Build**

1. Create an Expo account: **expo.dev/signup**

2. Install EAS CLI

```
npm install -g eas-cli
```

3. Log in to your Expo account

```
eas login
```

4. Verify login

```
eas whoami
```

Create Expo App

```
npx create-expo-app --template blank-typescript
```

Edit App.tsx

my-app/App.tsx

```
1 import { StyleSheet, Text, View } from "react-native";
2
3 export default function App() {
4   return (
5     <View style={styles.container}>
6       <Text style={styles.text}>Hello from EAS Build!</Text>
7     </View>
8   );
9 }
10
11 const styles = StyleSheet.create({
12   container: {
13     flex: 1,
14     backgroundColor: "#fff",
15     alignItems: "center",
16     justifyContent: "center",
17   },
18   text: {
19     fontSize: 30,
```

EAS Build: Setup

Configure your iOS/Android project for EAS Build

```
eas build:configure
```

- Create a `eas.json` file with build profiles: `development`, `preview`, `production`
- `eas.json` tells Expo how to build your app for a specific purpose

eas.json: Build Profiles

Each profile defines a set of build configuration options

```
{
  "cli": {
    "version": ">= 16.26.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {}
  }
}
```

- **"developmentClient": true**: Build a custom development client

```
npx expo install expo-dev-client
```

Custom Dev Client

A special version of your app that includes

- All **native modules** your app depends on
- JavaScript code + assets

Purpose: Test native modules and features Expo Go cannot run

- Expo Go cannot run apps with unsupported native modules

```
npx expo start --dev-client
```

Expo Go vs. Dev Client

Feature	Expo Go	Custom Dev Client
Modules included	Prebuilt (limited)	All modules your app depends on
Native modules	No test custom native code	Full support for custom native code
Use case	Quick testing / JS-only apps	Testing apps with custom native modules

- **Expo Go:** Prebuilt playground (limited modules)
- **Custom Development Client:** Your own playground with any toys you want (custom native modules)

eas.json: Build Profiles

```
{
  "cli": {
    "version": ">= 16.26.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {}
  }
}
```

- `distribution: "internal"` for internal testing, `store` for production release
- `autoIncrement`: Automatically increase version/build number for production builds

Build Profiles

- **development:** Fast builds for testing locally or on a few devices
- **preview:** Internal builds for sharing with teammates
- **production:** Final builds ready for App Store or Google Play

`eas.json` lets you switch between these build modes easily, without changing source code

```
eas build --profile <profile-name> --platform <android|ios>
```

Development Build

Create a testable build on device/emulator

For iOS

```
eas build --profile development --platform ios
```

For Android

```
eas build --profile development --platform android
```

EAS Build for iOS

```
eas build --profile development --platform ios
```

EAS Build for iOS requires **Apple certificates**

- **Development Certificate:** Allow installing and testing on physical devices
- **Distribution Certificate:** Allow submitting to TestFlight or App Store

Creating certificates requires **Apple Developer Program** membership (\$99 USD/year)

- Certificates themselves are not separately paid

Development Build

```
eas build --profile development --platform android
```

Android Keystore

A security file that contains your app's **signing key**

Every Android app installed on a device or submitted to the Play Store must be signed with a key from a keystore, which ensures:

- The app's authenticity (proves it comes from you)
- Users can safely install updates (they must be signed with the same key)

Development Build

APK vs. AAB

- **APK (Android Package):** Can be installed directly on devices, used for testing or internal distribution
- **AAB (Android App Bundle):** Required for publishing to Google Play Store
Development builds use APK
- Meant for testing locally or on a few devices
- APK is simple: download and install it immediately
- AAB is only required for production distribution

Build AAB with EAS

```
eas.json
```

```
"production": {  
  "gradleCommand": ":app:bundleRelease",  
  "distribution": "store"  
}
```

- `gradleCommand` specifies the Gradle (build system that Android uses to compile apps) task to run
- `:app`: The `app` module in your Android project
- `bundleRelease`: Build a release AAB for Google Play
- `assembleRelease`: Build production APK for manual distribution

```
eas build --profile production --platform android
```

Development Build

```
eas build --profile development --platform android
```

- Install on device via QR code or APK file
- Use with `npx expo start --dev-client`
- Ideal for testing native modules without Expo Go limitations

Preview Build

For internal testing/sharing

```
eas build --profile preview --platform all
```

- Generates shareable URL/QR for installation
- **iOS:** Up to 100 internal testers
 - Expo can automate device registration (`eas device:create`)
- **Android:** Unlimited testers
 - Direct install via APK

Production Build

Ready for app stores

```
eas build --profile production --platform all
```

- Optimize the app for release
- Remove unused code, minify JS, disable debug tools, etc.
- Handle signing credentials (iOS certificates, Android keystore) securely
- Outputs **.aab** (Android) or **.ipa** (iOS) for submission

Recap: EAS Build

Cloud-based native compilation for Expo and React Native apps

- No local Xcode or Android Studio required
Compiles your app into installable binaries
- iOS: `.ipa`
- Android: `.apk` (testing) or `.aab` (Google Play)

```
eas build:configure
```

- Creates `eas.json` with build profiles: `development`, `preview`, `production`

Recap: EAS Build

Build Commands

- **development:** Fast builds for testing locally or on a few devices

```
eas build --profile development --platform <ios/android>
```

- **preview:** Internal builds for sharing with teammates

```
eas build --profile preview --platform all
```

- **production:** Final builds ready for App Store or Google Play

```
eas build --profile production --platform all
```

Publish to Stores

Apple App Store

- Requires Apple Developer Program (\$99/year)

Google Play Store

- Requires a Google Play Developer Account (\$25 one-time)
- Use **EAS Submit** for automation after building

App Store Process

Official Doc: **Submit to the Apple App Store**

- . Enroll in Apple Developer Program
- ?. Create App ID and set up metadata in **App Store Connect**

App Store Connect

Apple's web portal for managing apps on the App Store

- Create and manage App IDs
- Configure app metadata (name, description, screenshots, categories, privacy info)
- Manage TestFlight builds for beta testing
- **TestFlight**: Apple's beta testing platform for iOS apps
- Allow developers to test their app on real devices before release
- Submit apps to Apple for review and publishing

App Store Process

1. Enroll in Apple Developer Program
2. Create App ID and set up metadata in **App Store Connect**
3. Build `.ipa` with EAS Build

```
eas build --profile production --platform ios
```

4. Submit

```
eas submit --platform ios
```

5. Review: 1-2 days (faster for updates)
- Common rejections: Crashes, privacy issues, incomplete metadata

EAS Workflows CI/CD

Use **EAS Workflows** to build and submit your app automatically

CI/CD

A software development practice to **automate** integration and delivery

CI: Continuous Integration

- Developers **commit code frequently**
- Each commit triggers **automatic build and test**
- Detects conflicts and errors early → smoother integration

CI/CD

CI: Continuous Integration

CD: Continuous Delivery / Continuous Deployment

- **Continuous Delivery:** Automatically **package and prepare** tested code for release
- Deployment to production still requires **manual approval**
- **Continuous Deployment:** Fully automated
- New versions are **deployed to production automatically** once tests pass

CI/CD

CI (Continuous Integration): Automatically **build and test** code whenever changes are made

CD (Continuous Delivery/Deployment): Automatically **release** tested code to staging or production

Key idea: CI/CD replaces slow, manual workflows with **frequent commits, automatic testing, and automated delivery**

→ Faster and more reliable software development

CI/CD Tool

GitHub Actions: Integrated CI/CD for GitHub repositories

- `yml` file in `.github/workflows/`

`.github/workflows/ci-cd.yml`

```
1 name: CI/CD Demo
2
3 on:
4   push:
5     branches:
6       - main
7   pull_request:
8     branches:
9       - main
10
11 jobs:
12   ci:
13     name: Continuous Integration
14     runs-on: ubuntu-latest
15     steps:
16       - uses: actions/checkout@v4
17       - uses: actions/setup-node@v4
18         with:
19           node-version: 22
```

EAS Workflows CI/CD

Use **EAS Workflows** to build and submit your app automatically

Create a workflow file at the root of your project

```
| .eas/workflows/submit-ios.yml
```

```
1 on:
2   push:
3     branches: ['main']
4
5 jobs:
6   build_ios:
7     name: Build iOS app
8     type: build
9     params:
10      platform: ios
11      profile: production
12
13   submit_ios:
14     name: Submit to TestFlight
15     needs: [build_ios]
16     type: testflight
17     params:
18      build_id: ${ needs.build_ios.outputs.build_id }
```


Google Play Process

Official Doc: [Submit to the Google Play Store](#)

1. Create **Google Play Console** account
2. Set up app in console (package name, icons, metadata)
3. Build **.aab** with EAS Build

```
eas build --profile production --platform android
```

4. Submit the build to Google Play

```
eas submit --platform android
```

5. Review: Hours to a few days

Tracks

Google Play allows you to release your app in different “tracks” to control who sees which version

- **Internal Testing:** Quick testing with your team
 - Up to 100 testers, very fast rollout
- **Closed alpha/beta:** Wider testing before production
 - Limited group of external testers (emails or opt-in link)
- **Production:** Official public release
 - Anyone on Google Play

EAS Workflows CI/CD

Use **EAS Workflows** to build and submit your app automatically

```
.eas/workflows/submit-android.yml
```

```
1 on:
2   push:
3     branches: ['main']
4
5 jobs:
6   build_android:
7     name: Build Android app
8     type: build
9     params:
10      platform: android
11      profile: production
12
13   submit_android:
14     name: Submit to Google Play Store
15     needs: [build_android]
16     type: submit
17     params:
18      platform: android
19      build_id: ${ needs.build_android.outputs.build_id }
```

Recap: EAS Submit

Automate the app store submission process after building your app with EAS Build

- Handle uploading your `.ipa` (iOS) or `.aab` (Android) automatically
- Git push → EAS Build → Build completes → EAS Submit → App Store / Play Store
- Work with store credentials / API keys you provide once
- Save time compared to manual submission and reduces the chance of errors

Pipeline

•

EAS Update

EAS Update: Push updates directly to users

- Work after your app is installed (App Store/Google Play)
- Let you push JavaScript and asset updates (not native code)
- No app store review required
- Updates are hosted on Expo's Content Delivery Network (CDN), not on the build server or app store
- Users automatically receive the latest JS bundle when they reopen app

EAS Update

Expo does **not** need to push updates to Apple App Store or Google Play when using **EAS Update**

When you use **EAS Build**, you create a **native shell**

- **Native shell:** A binary ([.ipa/.apk/.aab](#)) that wraps your React Native (JavaScript + assets) code, which includes
 - A small piece of native code
 - Logic to fetch the latest JS/asset bundle from Expo's Content Delivery Network (CDN)

EAS Update

Expo does **not** need to push updates to Apple App Store or Google Play when using **EAS Update**

When you use **EAS Build**, you create a **native shell**

Once your app is approved and installed from Apple App Store / Google Play Store, it can

- Check Expo's CDN for updates to your JS/asset bundle
- Download and apply updates automatically, without resubmission

EAS Update: Basic Usage

. Configure EAS Update

```
eas update:configure
```

Update `app.json` with `runtimeVersion` and `updates.url` properties

runtimeVersion

Specifies which version of your app a JavaScript bundle is compatible with

app.json

```
"runtimeVersion": {  
  "policy": "appVersion"  
}
```

"policy": "appVersion": Runtime version automatically matches app's version (`expo.version` in `app.json`)

- Updates are delivered **only** to devices with the same app version

runtimeVersion

Specifies which version of your app a JavaScript bundle is compatible with

app.json

```
"runtimeVersion": {  
  "policy": "appVersion"  
}
```

"policy": "appVersion": Runtime version automatically matches app's version (`expo.version` in `app.json`)

- **Prevents incompatible updates:** If you release a new native build (e.g., adding a native module), devices running older builds won't receive JS updates that depend on new native code

runtimeVersion

Specifies which version of your app a JavaScript bundle is compatible with

app.json

```
"runtimeVersion": {  
  "policy": "appVersion"  
}
```

"policy": "appVersion": Runtime version automatically matches app's version (`expo.version` in `app.json`)

"policy": "1.0.1": Useful if you want more control over which builds receive updates

EAS Update: Basic Usage

. Configure EAS Update

```
eas update:configure
```

Update `app.json` with `runtimeVersion` and `updates.url` properties

Update `eas.json` with setting the update `channel` property

Channels

eas.json

```
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "channel": "development"
    },
    "preview": {
      "distribution": "internal",
      "channel": "preview"
    },
    "production": {
      "autoIncrement": true,
      "channel": "production"
    }
  }
}
```

Allow you to point updates at specific types of builds

- Example: Publish updates to a preview build without impacting your production deployment

EAS Update: Basic Usage

1. Configure EAS Update
2. Configure the update channel
3. Publish an update with `eas update`

```
eas update --channel [channel-name] --message "[message]"
```

- `message`: Describe the update
- Send new JS bundle to the cloud
- Users get updated code automatically next time they open the app

EAS Update: Limitation

Can **only update JavaScript and static assets**

- Cannot modify native modules or configuration that require a rebuild

If you:

- Add a new native dependency (e.g., camera library)
- Change permissions, icons (native app's metadata), etc.

Then you must rebuild with EAS Build and resubmit via EAS Submit

Recap: EAS Update

Deliver new JS code and assets **without rebuilding or resubmitting** your app

- Works after your app is installed from app stores
- Uses Expo's CDN to host updates
- The app's native shell (built with EAS Build) automatically checks for updates
- Safe updates only — tied to compatible `runtimeVersion`
- Channels control which builds receive which updates

Limitation: Can update JavaScript and static assets only

EAS Services

EAS Build creates the “container” (native app shell)

EAS Submit delivers the “container” to the app stores

EAS Update sends new “contents” (JS/asset bundle) directly to users

- No re-delivery through the app stores

Developer → EAS Build → EAS Submit → App Store → Users → EAS Update

Recap: EAS

Service	Purpose	Runs Where	Produces
EAS Build	Builds native binaries	Expo's cloud servers	.apk , .aab , .ipa files
EAS Submit	Submits those binaries	Expo's cloud servers → app stores	Published app version
EAS Update	Pushes JS/asset updates to installs	Expo CDN	Updates live apps (no store resubmission)

Deploy App with Expo

- . Develop app with Expo + React Native
- ?. Build a native shell using `eas build`
- }. Test app on a device / emulator or share a preview build
- l. Option 1: Share a testable build (APK, IPA, or Expo Go link) – sufficient for course project
- ï. Option 2: Publish to App Store / Play Store (requires developer accounts)
- ð. Later: Push over-the-air fixes or tweaks with `eas update`

Course Summary

- **Styling and UI Design:** StyleSheet, Flexbox
- **Navigation:** React Navigation, Expo Router
- **State Management:** Context API, [useReducer](#), Redux Toolkit
- **Data Persistence:** React Native Async Storage
- **Notifications:** Local notifications, push notifications
- **Backend Integration:** Supabase
- **User Authentication:** Supabase
- **Deployment:** Expo Application Services (EAS)

Project Timeline

Milestone	Due Date
Presentation Slides	November 17
Project Introduction	November 17
Presentation	November 19 & 26
Final Project Deliverable	December 7

Presentation

Lecture 11 (November 19) Presentation Slots

Lecture 12 (November 26) Presentation Slots

Rubric

Presentation Rubric

- Clarity of Presentation (0-4 Points)
- Core Technical Requirements (0-12 Points)
- Advanced Features and Future Plan (0-4 Points)

Logistics

Presentation Logistics

- Arrive by 13:05; class starts at 13:10
- Next three teams sit in the front row
- 5 minutes presenting + 1 minute switching, strictly enforced
- Hand signal at 1 minute remaining
- Submit 70–100 word introduction to Quercus by Nov 17
- Begin directly with demo + technical highlights

Logistics

- One laptop, prepared in advance
- Classroom supports USB-C and HDMI
- Bring your own adapter if needed
- Live demo on emulator preferred
- Short clips allowed for network-dependent or device-specific features
- No Q&A and no interruptions
- If something is unclear, reflect in your peer evaluation
- All members attend both days for peer review

Speaker notes